

Rapport de projet C-Wildwater



Niveau : Pré-ing 2 MI

Groupe de projet : MI1_F :

- BELGHAZI Sara
- BOUDJEBINA Shems
- RGUIG Taissir

Année universitaire : 2025/2026

Introduction :

Ce rapport présente l'organisation de notre équipe, le déroulement de notre projet, les difficultés rencontrées et les solutions apportées, ainsi que des exemples d'exécution de notre travail.

Répartition des tâches au sein du groupe :

Dès le départ, nous avons participé de manière collaborative à chacune des étapes du projet, et notamment à sa vérification finale. Cependant, afin de travailler de manière efficace et organisée, nous nous sommes ensuite répartis ces tâches :

- **Shems BOUDJEBINA** : développement du programme en langage C.
- **Sara BELGHAZI** : écriture du script Shell et rédaction du rapport.
- **Taissir RGUIG** : réalisation du Makefile et rédaction du Readme.

Processus de réalisation du projet :

Une fois les premières versions des fichiers créées, nous avons effectué des premiers tests qui nous ont permis d'identifier plusieurs erreurs de syntaxe dans le programme en langage C et dans le script Shell. En effet, il manquait un point-virgule et un espace a été oublié entre un texte et un crochet dans une condition « if ».

Après avoir corrigé ces erreurs, nous avons lancé une seconde série de tests, et le terminal affichait encore plusieurs « warnings » ainsi qu'une erreur de segmentation. Cette dernière nous a posé des difficultés et a nécessité plusieurs essais et analyses avant d'en identifier l'origine. L'erreur provenait finalement d'une portion du fichier « script.sh » (plus tard renommé « run.sh ») plus précisément au niveau de la partie d'exécution du programme. Cette dernière a donc été modifiée puis retravaillée afin de corriger le problème.

Une fois ces erreurs résolues, nous avons enfin réussi à générer correctement les histogrammes, puis dans un second temps, à calculer et afficher les fuites (leaks).

Par la suite, nous avons relu et revérifié ensemble l'ensemble des fichiers.c pour améliorer la lisibilité et la compréhension du code. Pour cela, nous avons ajouté des commentaires aux endroits nécessaires et harmonisé les éléments affichés (ceux visibles par l'utilisateur) en les mettant en français, notamment les titres des histogrammes, les noms des fichiers de sortie, les messages d'erreur et les messages d'information.

Pour finir, Shems a pris l'initiative d'essayer d'implémenter quelques nouvelles lignes de code dans les fichiers.c et le script shell (run .sh) pour faire **la partie bonus**. Nous avons donc vérifié ensemble le fonctionnement du programme et corrigé les erreurs puis nous avons effectué les dernières modifications nécessaires (commentaires et affichage).

Planning de travail :

Semaine du 1er décembre :

- Compréhension de la consigne
- Répartition des tâches
- Création d'un dépôt sur Github
- Début de l'écriture du programme en langage C et du script shell

Semaine du 8 décembre :

- Premiers tests
- Correction des premières erreurs
- Modification du script shell

Semaine du 15 décembre :

- Correction des codes
- Amélioration de la présentation du code en renommant les fichiers.dat, les fichiers.png et les titres des images générés en français et en ajoutant des commentaires
- Rédaction du rapport et du readme
- Implémentation de nouvelles lignes de code pour ajouter la partie bonus

Exemples d'exécution :

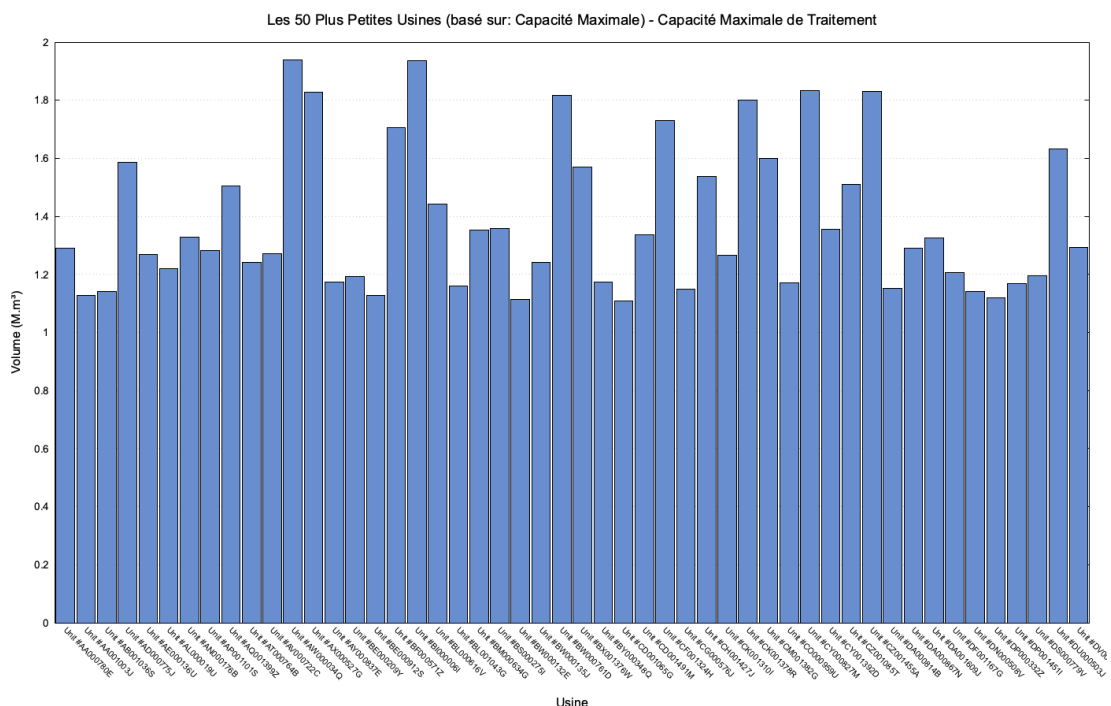
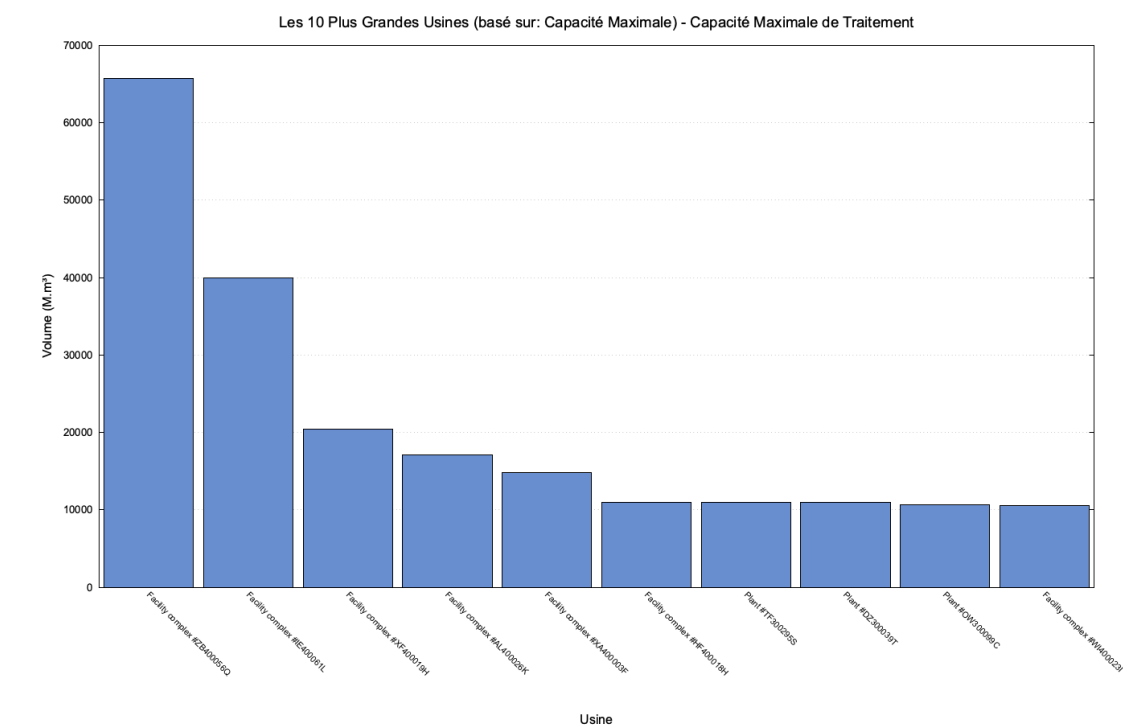
Nous avons créé deux branches sur Github : l'une qui se nomme « **main** », et qui contient tous les fichiers du projet sans bonus, et une autre qui se nomme « **Bonus** », et qui regroupe tous les fichiers du projet avec des lignes de code supplémentaires.

A noter que nous n'avons testé le programme « Bonus » que sur Mac et non sur Linux. C'est pourquoi la branche « main » reste fonctionnelle, si des problèmes venaient à apparaître lors de l'exécution du programme « Bonus ».

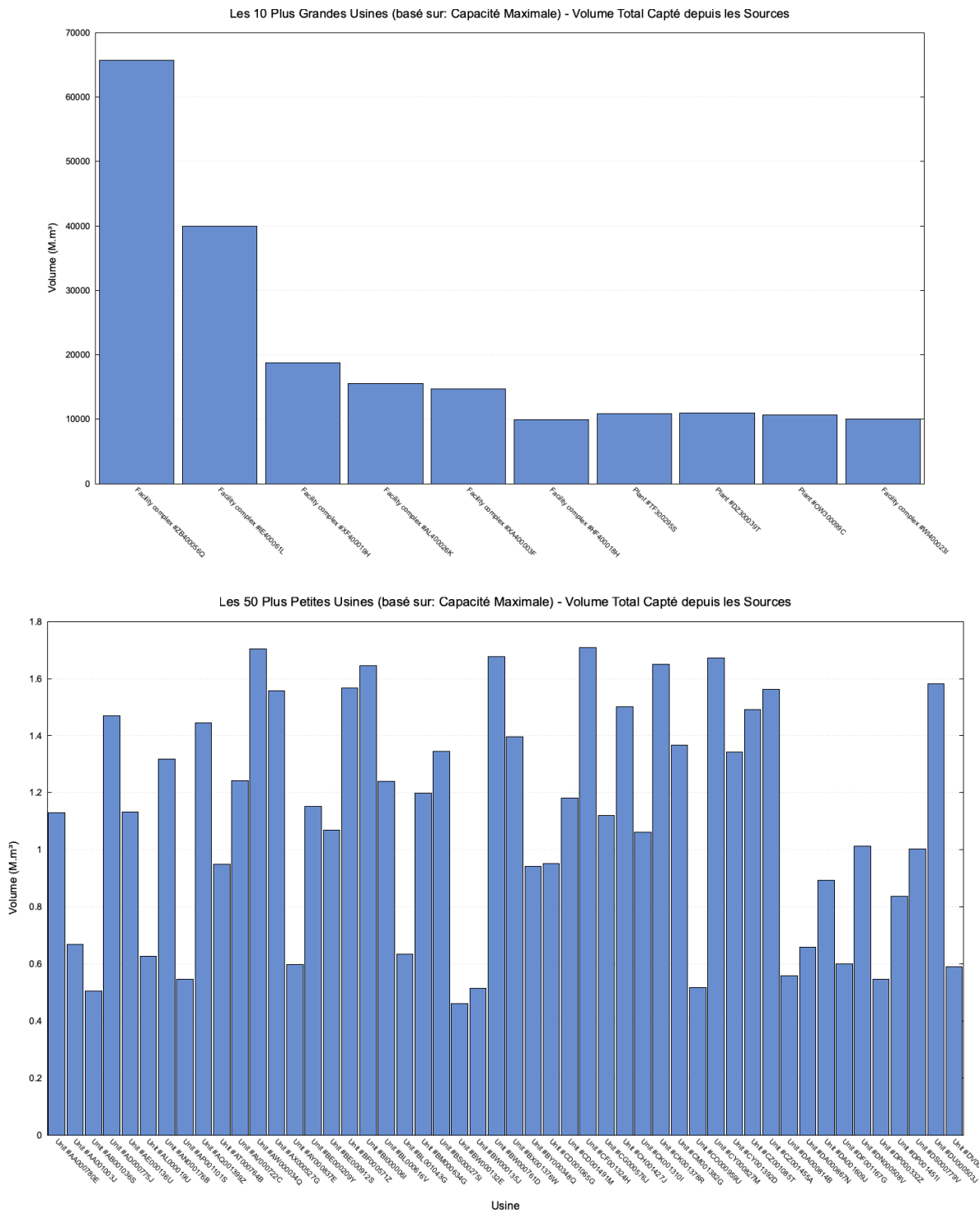
Pour la branche main (Sans bonus) :

Génération des histogrammes :

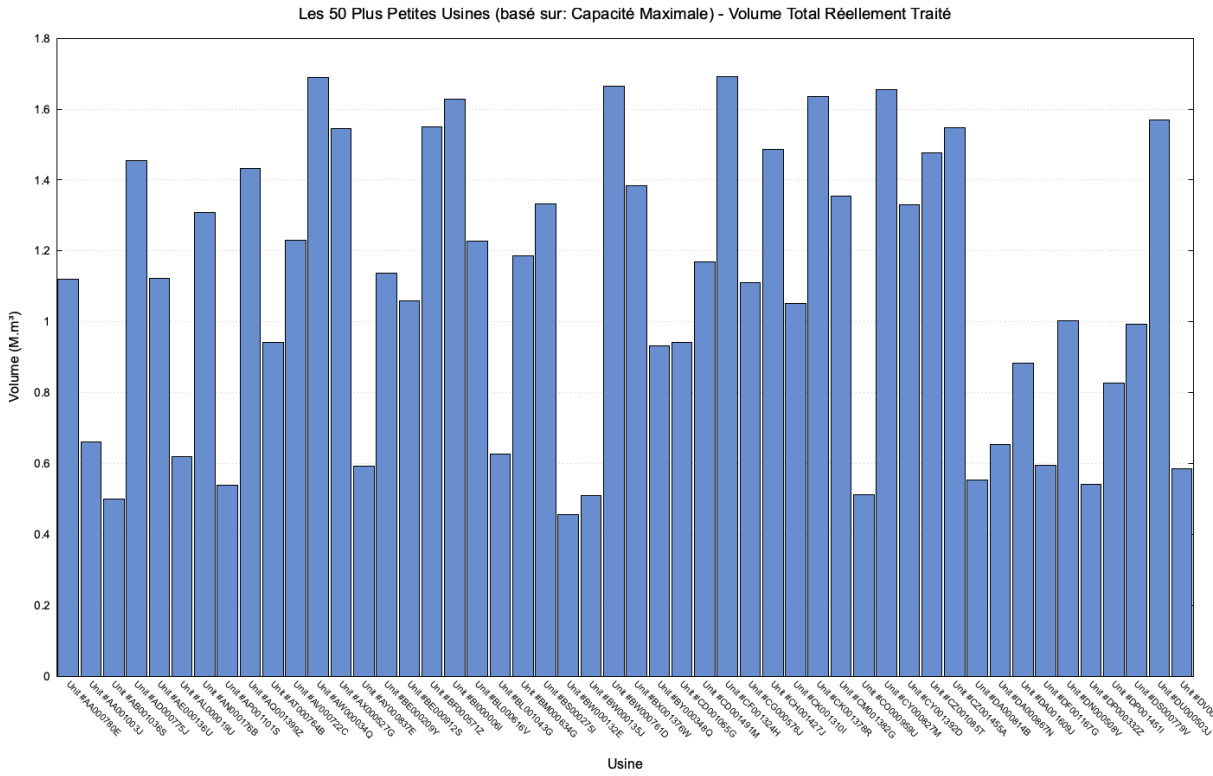
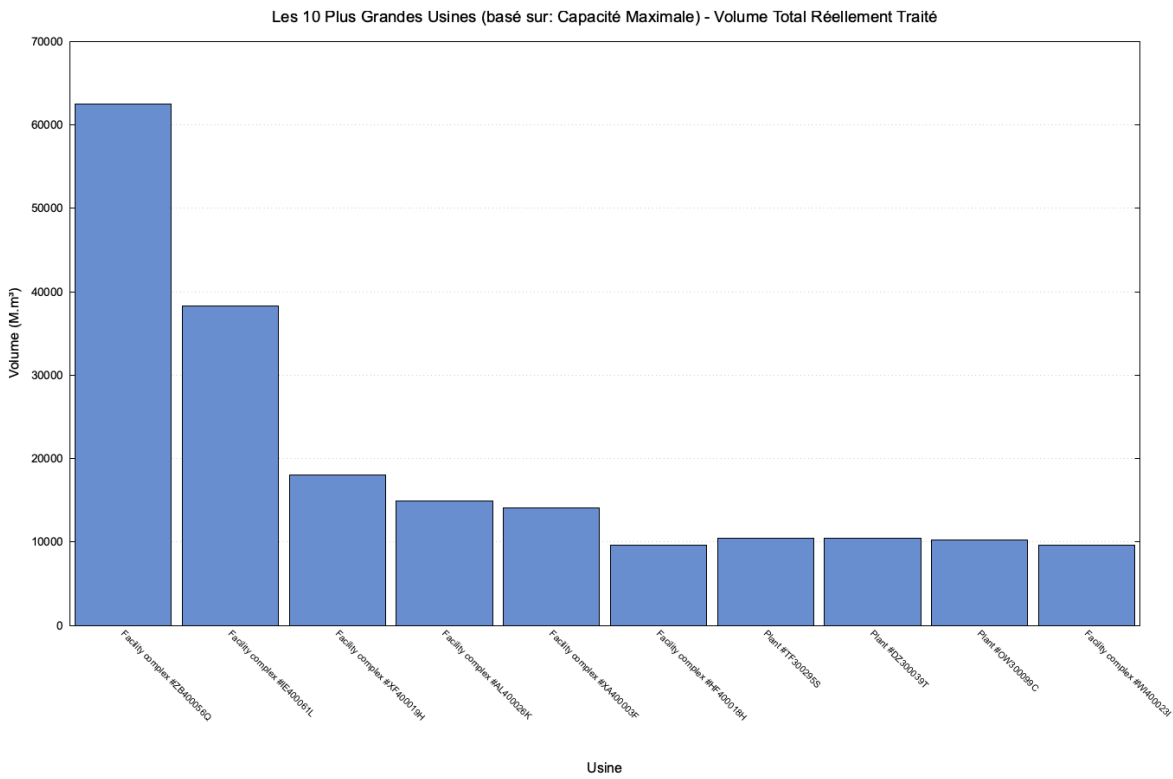
Histo max :



Histo src :



Histo real :



Calcul des fuites (leaks) :

Affichage du terminal :

```
shems.boudjebina@Air-de-Shems Projet-final % ./run.sh c-wildwater_v3.dat leaks "Facility complex #0C400007T"
Calcul des fuites pour l'usine: Facility complex #0C400007T
Volume total de fuites: 755.490272 M.m³/an
Résultat enregistré dans: leaks.dat
Traitement terminé avec succès.
Durée totale du script: 1000 ms
shems.boudjebina@Air-de-Shems Projet-final %
```

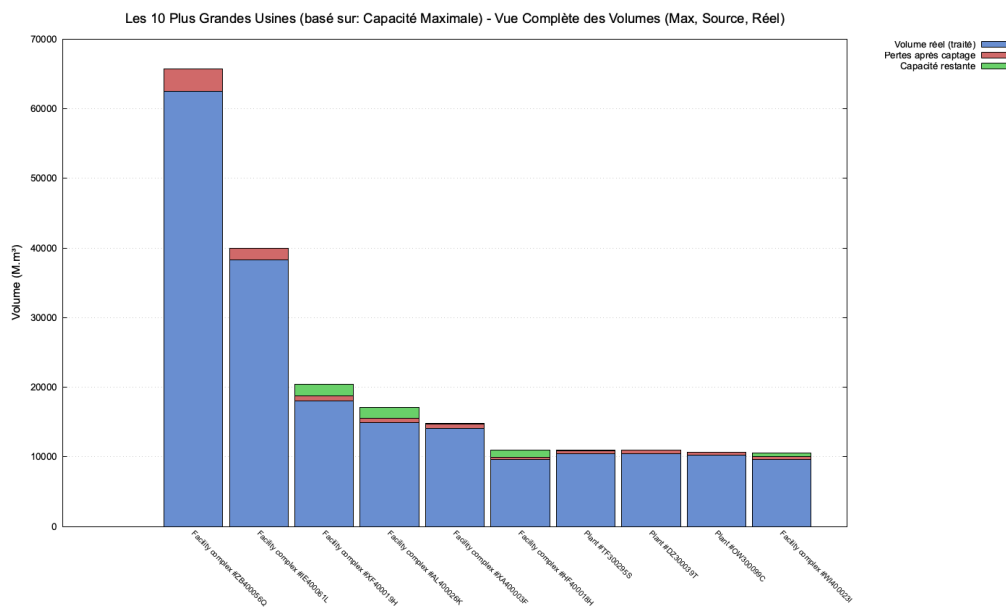
Ecriture dans le fichier « leaks.dat » :

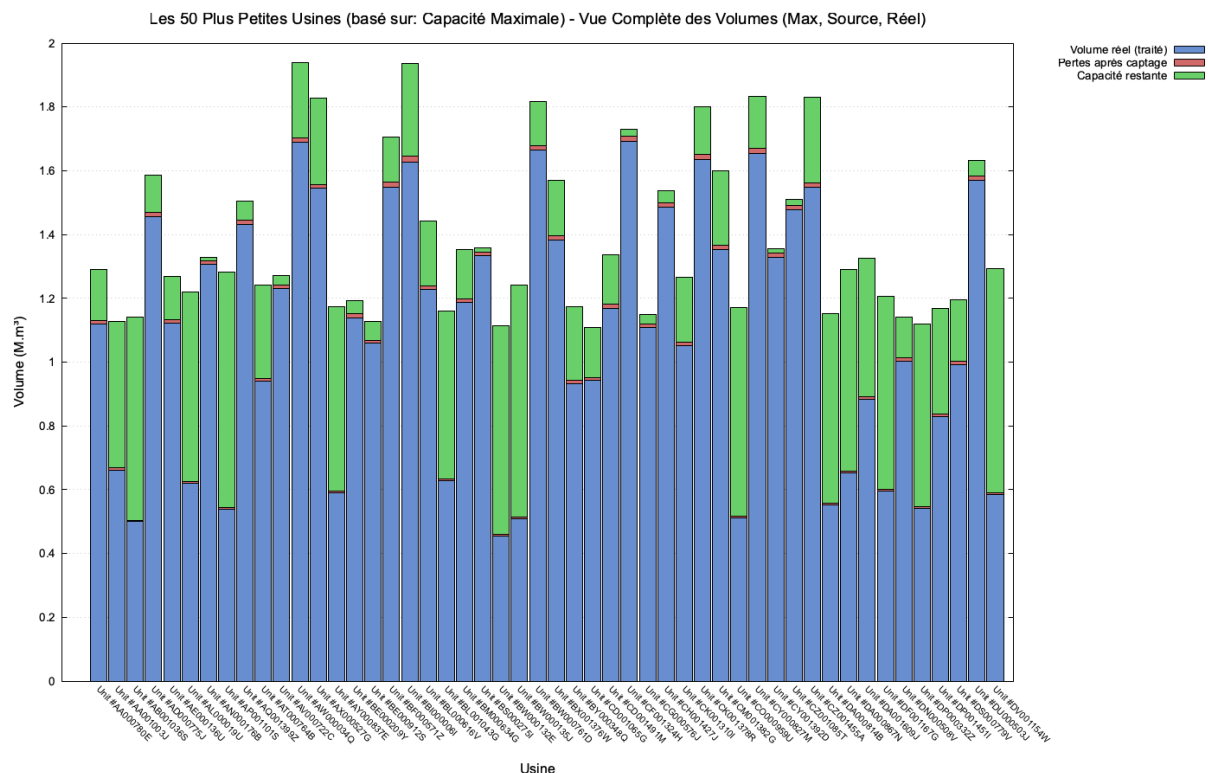


Pour la branche Bonus :

Génération de l'histogramme regroupant les trois volumes (max/src/real) :

Histo all :





Calcul des fuites (leaks) :

Affichage du terminal :

```
shems.boudjebina@Air-de-Shems Bonus-final % ./run.sh c-wildwater_v3.dat leaks "Facility complex #0C400007T"
Calcul des fuites pour l'usine: Facility complex #0C400007T
Volume total de fuites: 755.490272 M.m³/an
Pire tronçon: Storage #11085 -> Junction #FF11085Y (40.014900 M.m³/an)
Résultat enregistré dans: leaks.dat
Traitement terminé avec succès.
Durée totale du script: 1000 ms
```

Ecriture dans le fichier « leaks.dat » :

```
Identifiant;volume fuites (M.m³.an-1);tronçon pire amont;tronçon pire aval;volume fuite pire tronçon (M.m³.an-1)
Facility complex #0C400007T;755.490272;Storage #11085;Junction #FF11085Y;40.014900
Module #CM101143F;15.892287;Storage #02790;Junction #NP02790L;1.873016
```

Limitations fonctionnelles :

Au démarrage du projet, un fichier « histo.dat » a été créé lors de l'exécution du script (run.sh). Mais après réflexion, nous avons finalement préféré opter pour la création de trois fichiers distincts, spécifiques à chaque type d'histogramme : « max_volume.dat » ; « src_volume.dat » ; « real_volume.dat » (et « all_volume.dat » pour la partie bonus), et ce, après modification du script (run.sh).

Bien que ce premier fichier « histo.dat » n'ait aucun impact fonctionnel, nous avons tout de même souhaité le conserver de peur que sa suppression n'affecte l'exécution du programme.